

SAIJ MOHAMMED

TP6 - Optimisation des performances (DWH SH)

Objectifs

- analyser les performances d'une requête
- utiliser différents **types d'index**
- comprendre le rôle du **CBO (Cost Based Optimizer)**
- collecter et exploiter les **statistiques**
- introduire le **partitionnement**
- lire un **plan d'exécution**

1. Contexte

Vous travaillez sur un **Data Warehouse basé sur SH** : (vous faite une copie du schéma SH)

Tables principales :

- SALES
- CUSTOMERS
- PRODUCTS
- TIMES
- CHANNELS

2. Travaux demandés

Partie A - Analyse initiale

1. Exécuter une requête analytique :
 - ventes par mois
 - ventes par produit
 - ventes par région

```
SQL> select d.department_name,
        count(*) nb_employees
from employees e
join departments d
  on e.department_id = d.department_id
group by d.department_name
order by nb_employees desc; 2    3    4    5    6    7

DEPARTMENT_NAME          NB_EMPLOYEES
-----
Shipping                  45
Sales                     34
Purchasing                6
Finance                   6
IT                         5
Executive                 3
Accounting                2
Marketing                 2
Administration            1
Public Relations          1
Human Resources           1

11 rows selected.
```

2. Observer :

- temps d'exécution
- volume de données

```
SQL> EXPLAIN PLAN FOR
```

```
select d.department_name,
        count(*) nb_employees
from employees e
join departments d
  on e.department_id = d.department_id
group by d.department_name
order by nb_employees desc;
```

```
select * from table(dbms_xplan.display); 2    3    4    5    6    7    8
```

Explained.

```
SQL> SQL>

PLAN_TABLE_OUTPUT
-----
Plan hash value: 2234722717

-----
-----
| Id | Operation          | Name          | Rows  | Bytes | Cost (%CPU)| Time
|-----|-----|-----|-----|-----|-----|-----|
| 0 | SELECT STATEMENT    |               | 27    | 594   | 7  (15) | 00:00
:01 |
| 1 | SORT ORDER BY       |               | 27    | 594   | 7  (15) | 00:00
:01 |
| 2 | HASH GROUP BY       |               | 27    | 594   | 7  (15) | 00:00
:01 |
|* 3 | HASH JOIN           |               | 106   | 2332  | 6   (0) | 00:00
:01 |
PLAN_TABLE_OUTPUT
-----
| 4 | VIEW                | VW_GBF_7      | 27    | 513   | 3   (0) | 00:00
:01 |
| 5 | TABLE ACCESS FULL | DEPARTMENTS   | 27    | 432   | 3   (0) | 00:00
:01 |
| 6 | TABLE ACCESS FULL | EMPLOYEES      | 107   | 321   | 3   (0) | 00:00
:01 |
-----
PLAN_TABLE_OUTPUT
-----
```

Partie B - Index

Créer des index adaptés :

- sur les clés de jointure
create index IDX_EMP_BIG_DEPT
on EMP_BIG(DEPARTMENT_ID);
- sur les colonnes de filtrage
create index IDX_EMP_BIG_EMP
on EMP_BIG(EMPLOYEE_ID);
- sur les colonnes de regroupement
create index IDX_EMP_JOB

on EMPLOYEES(JOB_ID);

INDEX_NAME

TABLE_NAME

INDEX_TYPE

COUNTRIES_PK

COUNTRIES

NORMAL

DEPARTMENTS_PK

DEPARTMENTS

NORMAL

INDEX_NAME

TABLE_NAME

INDEX_TYPE

EMPLOYEES_PK

EMPLOYEES

NORMAL

IDX_EMP_BIG_DEPT

EMP_BIG

INDEX_NAME

TABLE_NAME

INDEX_TYPE

NORMAL

IDX_EMP_BIG_EMP

EMP_BIG

NORMAL

JOBS_PK

```

JOBS_PK
INDEX_NAME
-----
TABLE_NAME
-----
INDEX_TYPE
-----
JOBS
NORMAL
-LOCATIONS_PK
LOCATIONS
NORMAL

INDEX_NAME
-----
TABLE_NAME
-----
INDEX_TYPE
-----
REGIONS_PK
REGIONS
NORMAL

8 rows selected.

```

Comparer :

- **B-tree**
- **Bitmap**

Les index B-tree et Bitmap sont utilisés pour améliorer les performances des requêtes SQL, mais ils sont adaptés à des contextes différents. Les index B-tree sont les plus utilisés dans les systèmes transactionnels (OLTP) et conviennent aux colonnes à forte cardinalité contenant beaucoup de valeurs distinctes, comme employee_id ou email. Ils sont efficaces pour les recherches utilisant =, <, > ou BETWEEN. En revanche, les index Bitmap sont principalement utilisés dans les Data Warehouses pour les requêtes analytiques. Ils sont adaptés aux colonnes à faible cardinalité, comme le genre ou la région, et permettent d'optimiser les requêtes comportant plusieurs filtres. Cependant, ils sont moins efficaces dans les environnements avec de nombreuses mises à jour.

Partie C - Statistiques CBO

- collecter les statistiques sur les tables

```

SQL> ^M
SQL> begin
  dbms_stats.gather_table_stats(
    ownname => 'HR_TP6',
    tabname => 'EMP_BIG',
    cascade => true
  );
end;
/ 2      3      4      5      6      7      8

PL/SQL procedure successfully completed.

SQL> █

```

- vérifier l'impact sur le plan d'exécution

```

SQL> EXPLAIN PLAN FOR
select *
from emp_big
where employee_id = 100;

select * from table(dbms_xplan.display); 2      3      4
Explained.

SQL> SQL>

PLAN_TABLE_OUTPUT
-----
Plan hash value: 1946431486

-----
| Id | Operation          | Name | Rows | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT   |      |  5000 |  336K |  1507   (1)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| EMP_BIG |  5000 |  336K |  1507   (1)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----

PLAN_TABLE_OUTPUT
-----

      1 - filter("EMPLOYEE_ID"=100)

13 rows selected.

SQL> █

```

Après la collecte des statistiques :

- Oracle estime plus précisément les coûts
- le CBO prend de meilleures décisions
- les plans d'exécution deviennent plus fiables
- Oracle peut décider si un index est réellement utile ou non

Partie D - Explain Plan

- générer le plan d'exécution

```
EXPLAIN PLAN FOR
select e.employee_id,
       e.first_name,
       d.department_name
from employees e
join departments d
  on e.department_id = d.department_id;
```

```
select * from table(dbms_xplan.display);
```

- analyser :

- Full Table Scan

Le Full Table Scan correspond à une lecture complète d'une table. Oracle parcourt toutes les lignes afin de trouver les données demandées. Cette méthode est souvent utilisée lorsque la table est petite ou lorsqu'aucun index sélectif n'est disponible. Dans le TP, Oracle a utilisé un TABLE ACCESS FULL sur la table EMPLOYEES.

- Index Scan

L'Index Scan permet à Oracle d'accéder rapidement aux lignes grâce à un index sans parcourir toute la table. Cette méthode réduit le coût des recherches sélectives. Dans le TP, Oracle a utilisé un INDEX FULL SCAN sur l'index DEPARTMENTS_PK.

- Hash Join

Le Hash Join est une méthode de jointure efficace pour les grands volumes de données et les conditions d'égalité. Oracle crée une table de hachage en mémoire afin d'associer rapidement les lignes des deux tables. Cette méthode est souvent utilisée dans les requêtes analytiques.

- Nested Loop

Le Nested Loop consiste à parcourir une première table puis rechercher les lignes correspondantes dans la seconde table. Cette méthode est performante lorsque le nombre de lignes retournées est faible et qu'un index existe sur la table jointe. Elle est très utilisée dans les systèmes transactionnels (OLTP).

Partie E - Partitionnement

Simuler un partitionnement sur SALES :

- par date

```
SQL>
SQL> create table SALES (
    sale_id      number,
    sale_date    date,
    region       varchar2(20),
    amount       number
); 2      3      4      5      6

Table created.

SQL> █
```

```
SQL> insert into SALES values (1, date '2024-01-10', 'NORTH', 1000);

1 row created.

SQL> insert into SALES values (2, date '2024-02-15', 'SOUTH', 1500);

1 row created.

SQL> insert into SALES values (3, date '2024-03-20', 'EAST', 2000);

1 row created.

SQL> insert into SALES values (4, date '2024-04-05', 'WEST', 1200);

1 row created.

SQL> commit;

Commit complete.
```

- ou par région

SALE_ID	SALE_DATE	REGION	AMOUNT
1	10-JAN-24	NORTH	1000
2	15-FEB-24	SOUTH	1500
3	20-MAR-24	EAST	2000
4	05-APR-24	WEST	1200

```
SQL> █
```



```

SQL> create table SALES_PART (
    sale_id      number,
    sale_date    date,
    region       varchar2(20),
    amount       number
)
partition by range (sale_date)
(
    partition p_q1 values less than (date '2024-04-01'),
    partition p_q2 values less than (date '2024-07-01')
);  2      3      4      5      6      7      8      9     10     11

Table created.

SQL> insert into SALES_PART
select * from SALES;  2

4 rows created.

SQL> commit;

Commit complete.

SQL> select table_name,
           partition_name
from user_tab_partitions
where table_name = 'SALES_PART';  2      3      4

TABLE_NAME
-----
PARTITION_NAME
-----
SALES_PART
P_Q1

SALES_PART
P_Q2

```

Comparer :

- performances avant / après

avant :

```

SQL> EXPLAIN PLAN FOR
select *
from SALES
where sale_date between date '2024-01-01'
                        and date '2024-03-31';

select * from table(dbms_xplan.display); 2      3      4      5
Explained.

SQL> SQL>

PLAN_TABLE_OUTPUT
-----
Plan hash value: 781590677

-----
| Id | Operation                | Name | Rows  | Bytes | Cost (%CPU)| Time     |
-----+-----+-----+-----+-----+-----+-----+
|  0 | SELECT STATEMENT         |      |      3 | 141   | 3   (0)| 00:00:01 |
|*  1 | TABLE ACCESS FULL      | SALES |      3 | 141   | 3   (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----

PLAN_TABLE_OUTPUT
-----

      1 - filter("SALE_DATE">=TO_DATE(' 2024-01-01 00:00:00', 'syyyy-mm-dd
            hh24:mi:ss') AND "SALE_DATE"<=TO_DATE(' 2024-03-31 00:00:00',
            'syyyy-mm-dd hh24:mi:ss'))

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)

19 rows selected.

```

Après :

```
SQL> EXPLAIN PLAN FOR
select *
from SALES_PART
where sale_date between date '2024-01-01'
                        and date '2024-03-31';

select * from table(dbms_xplan.display);  2      3      4      5
Explained.

SQL> SQL>
```

PLAN_TABLE_OUTPUT

Plan hash value: 3468116159

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
	Pstart Pstop					

PLAN_TABLE_OUTPUT

0	SELECT STATEMENT		1	47	274 (0)	00:00:01
1	PARTITION RANGE SINGLE		1	47	274 (0)	00:00:01
2	TABLE ACCESS FULL	SALES_PART	1	47	274 (0)	00:00:01

PLAN_TABLE_OUTPUT

```

PLAN_TABLE_OUTPUT
-----

Predicate Information (identified by operation id):
-----

      2 - filter("SALE_DATE">=TO_DATE(' 2024-01-01 00:00:00', 'syyyyy-mm-dd hh24:mi:ss') AND

              "SALE_DATE"<=TO_DATE(' 2024-03-31 00:00:00', 'syyyyy-mm-dd hh24:mi:ss'))

PLAN_TABLE_OUTPUT
-----

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)

19 rows selected.

SQL> █

```

Avant le partitionnement, Oracle utilisait un TABLE ACCESS FULL sur la table SALES, ce qui signifie que toute la table était parcourue pour rechercher les lignes correspondant à la condition sur SALE_DATE. Le coût estimé observé était de 333. Après le partitionnement, Oracle utilise l'opération PARTITION RANGE SINGLE, ce qui montre que seule la partition P_Q1 est consultée grâce au mécanisme de Partition Pruning. Cela permet de limiter les lectures aux données nécessaires sans parcourir toutes les partitions. Même si le coût affiché dans cette petite simulation est plus élevé, avec une valeur de 274274274, le partitionnement devient très performant sur les grandes tables car il réduit considérablement le volume de données analysé par Oracle.

Partie F - Analyse

Répondre :

1. Pourquoi le bitmap index est adapté au DWH ?
Le Bitmap Index est particulièrement adapté aux Data Warehouses car il fonctionne très bien sur les colonnes ayant peu de valeurs distinctes, comme le sexe, la région ou le statut. Il permet d'accélérer fortement les requêtes analytiques contenant des filtres et des opérations de regroupement. Dans un environnement décisionnel, les données sont principalement lues et peu modifiées, ce qui correspond parfaitement au fonctionnement des Bitmap Index.
2. Pourquoi les statistiques sont essentielles ?
Les statistiques permettent à l'optimiseur Oracle (CBO) de choisir le meilleur plan d'exécution pour une requête. Elles fournissent des informations importantes comme le nombre de lignes, la taille des tables ou la distribution des données. Sans statistiques à jour, Oracle peut choisir un plan inefficace, entraînant des temps d'exécution plus élevés.
3. Différence entre Hash Join et Nested Loop ?
Le Hash Join est utilisé principalement pour les grands volumes de données. Oracle crée une table de hachage en mémoire afin de comparer rapidement les lignes des tables jointes. Cette méthode est efficace pour les traitements massifs.

Le Nested Loop fonctionne différemment : Oracle parcourt les lignes d'une table puis recherche les correspondances dans l'autre table, souvent via un index. Cette méthode est plus adaptée aux petites tables ou aux requêtes retournant peu de lignes.

4. Quel est l'intérêt du partitionnement ?

Le partitionnement permet de diviser une grande table en plusieurs partitions plus petites. Oracle peut alors accéder uniquement aux partitions nécessaires lors d'une requête grâce au mécanisme de Partition Pruning. Cela améliore les performances, réduit les temps d'accès aux données et facilite également l'administration et la maintenance des grandes tables.

5. Comment interpréter un plan d'exécution ?

Un plan d'exécution décrit les opérations utilisées par Oracle pour exécuter une requête SQL. Il permet d'identifier les méthodes d'accès aux données comme TABLE ACCESS FULL ou INDEX SCAN, ainsi que les types de jointures comme HASH JOIN ou NESTED LOOP. L'analyse du coût, du nombre de lignes estimées et des opérations effectuées permet de comprendre les performances d'une requête et d'identifier les optimisations possibles.

Livrables

- résultats des tests
- plans d'exécution
- analyse des résultats